

**PROGRAMA DE EXTENSÃO UFMS DIGITAL (95DX7.200525)**  
**AValiação DO MÓDULO 2 - DESENVOLVIMENTO WEB COM**  
**FRAMEWORKS E HTML/CSS**

|                                 |                                                                                          |
|---------------------------------|------------------------------------------------------------------------------------------|
| <b>Nome completo:</b>           | Matheus Leal da Gama                                                                     |
| <b>Disciplina:</b>              | Projeto Integrador de Tecnologia da Informação II                                        |
| <b>Semestre letivo / Curso:</b> | 2026/2 — Tecnologia da Informação                                                        |
| <b>Público-alvo:</b>            | Protetores independentes, projetos voluntários de proteção animal e potenciais adotantes |
| <b>Local de realização:</b>     | Brasília - DF                                                                            |
| <b>Título da ação:</b>          | PLATAFORMA WEB PARA APOIO À ADOÇÃO RESPONSÁVEL DE CÃES                                   |

**Resumo:** Este relatório apresenta o desenvolvimento completo do Módulo 2 do Projeto Integrador de Tecnologia da Informação II, focado na construção de uma aplicação web dinâmica e responsiva utilizando o framework Django (Python 5.2), HTML5 semântico, CSS3 moderno e JavaScript progressivo. A aplicação 'Patas de Brasília' visa resolver os problemas de pulverização de dados e falta de controle da situação dos cães resgatados por protetores independentes. A solução abrange a arquitetura Model-View-Template (MVT), autenticação de protetores, cadastro com mídias, filtros dinâmicos, integração com WhatsApp para contato direto com o responsável, seção de doações via PIX e registro automatizado de histórico de transição de status dos cães (RF01 a RF08; RNF01 a RNF04). O código-fonte foi versionado no Git e disponibilizado em repositório público no GitHub com instrução de instalação e testes automatizados.

**Palavras-chave:** Desenvolvimento Web. Framework Django. HTML5 e CSS3. Adoção Animal. Responsividade.

## **1 INTRODUÇÃO**

No cenário da proteção animal independente, pequenos grupos e voluntários enfrentam sérios desafios operacionais para manter organizados os dados dos cães resgatados sob sua tutela. Conforme identificado na etapa inicial do Módulo 1 deste projeto, a rotina desses protetores engloba a gestão de dezenas de animais simultaneamente, com registros pulverizados em conversas de aplicativos de mensagem (WhatsApp), histórias temporárias no Instagram, planilhas avulsas, cadernos e pastas de exames veterinários.

Essa descentralização das informações resulta em dados desatualizados, dificuldades na verificação rápida do responsável por cada cão, retrabalho na comunicação interna e perda de oportunidades de adoção responsável. Diante deste diagnóstico, o presente módulo contempla a construção de uma aplicação web dinâmica que consolida o Produto Mínimo Viável (MVP) do projeto 'Patás de Brasília'.

A aplicação foi estruturada a partir da arquitetura MVT (Model-View-Template) do framework Django, com interface desenvolvida segundo princípios de HTML5 semântico e acessível, além de estilização responsiva baseada em CSS3 puro mobile-first. O público-alvo beneficia-se diretamente de duas visões complementares: uma área pública simplificada e atraente para o público adotante e doadores, e uma área restrita e intuitiva para que os protetores autenticados possam gerenciar os cadastros e atualizar o histórico de cada animal.

## **2 OBJETIVO GERAL**

Desenvolver e implementar uma aplicação web dinâmica utilizando o framework Django, HTML5 semântico e CSS responsivo para auxiliar protetores independentes na organização, divulgação pública e acompanhamento contínuo da situação dos cães resgatados e disponíveis para adoção.

## **3 OBJETIVOS ESPECÍFICOS**

- Definir a estrutura da aplicação e seus componentes por meio do padrão arquitetural Model-View-Template (MVT) do Django.
- Implementar uma interface pública e interna responsiva com HTML5 semântico e CSS3 puro, adotando a abordagem mobile-first e diretrizes de acessibilidade.
- Configurar o framework web Django para gerenciar a autenticação de protetores, persistência relacional (SQLite), upload de imagens e versionamento do histórico de status dos animais.
- Desenvolver testes automatizados de integração e validar as regras de negócio essenciais da aplicação.
- Publicar o código-fonte em um repositório público no GitHub, devidamente documentado com orientações de execução e implantação no arquivo README.

## **4 JUSTIFICATIVA E DELIMITAÇÃO DO PROBLEMA**

A escolha do desenvolvimento web dinâmico fundamenta-se no fato de que protetores independentes e potenciais adotantes acessam informações predominantemente por meio de dispositivos móveis em deslocamento. A construção de uma plataforma centralizada elimina a perda de históricos causada pelo desaparecimento de publicações temporárias nas redes sociais e reduz o tempo necessário para responder a dúvidas sobre os animais.

A opção pelo framework Django justifica-se por sua alta produtividade, princípio 'baterias incluídas' (fornecendo autenticação segura, ORM expressivo, painel administrativo nativo e proteção automática contra vulnerabilidades como CSRF, SQL Injection e XSS), além de facilidade na expansão do código. Em relação à interface, optou-se por utilizar HTML5 e CSS3 sem frameworks CSS de mercado (como TailwindCSS ou Bootstrap) para garantir um código limpo, desempenho otimizado, total controle sobre a semântica acessível (WAI-ARIA) e facilidade de manutenção.

O escopo deste módulo delimita-se ao gerenciamento básico dos cães resgatados, divulgação pública, contato direto com o protetor responsável e apoio a doações via PIX. Módulos complexos de prontuário veterinário completo ou gestão financeira contábil avançada não fazem parte da presente entrega, mantendo o foco na usabilidade e agilidade requeridas pelo público-alvo.

## **5 FUNDAMENTAÇÃO TEÓRICA**

O desenvolvimento de aplicações web modernas fundamenta-se no desacoplamento entre modelo de dados, lógica de apresentação e interface com o usuário. Conforme destacam Pressman e Maxim (2019), a utilização de frameworks web estabelece uma infraestrutura reutilizável que padroniza rotas, tratamento de formulários e persistência em banco de dados, reduzindo sensivelmente a incidência de falhas estruturais.

No ecossistema Python, o Django adota o padrão MVT (Model-View-Template). Segundo a Django Software Foundation (2026), o 'Model' gerencia a estrutura e validações dos dados; a 'View' executa a regra de negócio e seleciona os dados pertinentes; e o 'Template' é responsável pela renderização da interface HTML final enviada ao navegador do cliente.

Para a interface com o usuário, o Design Web Responsivo (RWD) e a Semântica HTML são requisitos indispensáveis. Sommerville (2015) ressalta que a engenharia de software deve priorizar atributos de qualidade como usabilidade, portabilidade e acessibilidade. A estruturação semântica com elementos HTML5 (<header>, <main>, <nav>, <section>, <article>, <footer>) integrada às diretrizes do W3C/WCAG assegura que leitores de tela e tecnologias assistivas interpretem corretamente o conteúdo. Complementarmente, a abordagem mobile-first estabelece o fluxo de estilização CSS a partir de telas menores, expandindo o layout progressivamente via media queries (ISO/IEC 25010, 2017).

## 6 METODOLOGIA

O desenvolvimento do projeto seguiu uma abordagem incremental e iterativa, estruturada em três etapas principais: planejamento/definição, implementação técnica e publicação/documentação.

### 6.1 Planejamento e Definição do Projeto

A partir dos achados da entrevista realizada no Módulo 1, os requisitos funcionais (RF) e não funcionais (RNF) foram detalhados e priorizados para o MVP, conforme resumido na Tabela 1.

| Código | Requisito              | Descrição no Sistema                                                                    |
|--------|------------------------|-----------------------------------------------------------------------------------------|
| RF01   | Autenticação           | Login seguro para protetores responsáveis pelo gerenciamento.                           |
| RF02   | Cadastro de Cães       | Formulário completo com mídias (foto principal e adicionais), dados físicos e de saúde. |
| RF03   | Situação do Animal     | Controle de status (Tratamento, Lar Temporário, Disponível, Em Adoção, Adotado).        |
| RF04   | Vitrine Pública        | Página pública com filtros por sexo, porte, idade e busca por nome/região.              |
| RF05   | Filtro por Responsável | Painel interno com identificação do protetor e filtro 'Somente os meus'.                |
| RF06   | Contato Direto         | Botão de adoção que gera mensagem formatada direto no WhatsApp do protetor.             |
| RF07   | Apoio / PIX            | Seção de doações com chave PIX configurável e recurso de cópia rápida.                  |
| RF08   | Histórico de Status    | Rastreabilidade completa de mudanças de status com data, autor e observações.           |
| RNF01  | Responsividade         | Layout mobile-first adaptável a smartphones, tablets e computadores.                    |
| RNF02  | Usabilidade            | Interface intuitiva permitindo atualização de status em formulário simplificado.        |

|       |                  |                                                                                     |
|-------|------------------|-------------------------------------------------------------------------------------|
| RNF03 | Segurança        | Área administrativa restrita protegida por controle de sessão (LoginRequiredMixin). |
| RNF04 | Privacidade/LGPD | Coleta mínima de dados pessoais e restrição de acesso a observações internas.       |

## 6.2 Implementação

A implementação foi desenvolvida na linguagem Python 3.13 com Django 5.2. A arquitetura de arquivos seguiu a estrutura recomendada pelo framework:

- Modelo de dados (models.py): Criação das entidades Protetor (um-para-um com User), Cao (com escolhas enumeradas para sexo, porte, idade e status), FotoCao (um-para-muitos com Cao para fotos extras), HistoricoStatus (para rastreabilidade) e ConfiguracaoSite (padrão Singleton para parâmetros gerais e chave PIX).
- Regras de negócio e visões (views.py): Utilização de Class-Based Views (ListView, DetailView, CreateView, UpdateView, DeleteView) e Mixins de autenticação (LoginRequiredMixin) para garantir a segurança da área interna.
- Formatação e validação (forms.py): Criação de formulários personalizados para cadastro de cães com campo para múltiplas mídias, atualização rápida de status com validação da data de adoção e formulário de filtros dinâmicos.
- Interface visual (templates e static): Desenvolvimento de templates com HTML5 semântico e folha de estilos CSS3 modularizada com tokens CSS (variáveis HSL, grid flexível, tipografia Nunito/Fraunces) e arquivo JavaScript leve para suporte ao menu móvel, galeria e cópia de PIX.

## 6.3 Publicação e Documentação

O projeto foi organizado com controle de versão Git, ignorando arquivos temporários e bancos locais via .gitignore. A suíte de testes automatizados (animais/tests.py) foi elaborada contendo 9 testes unitários e de integração cobrindo autenticação, visibilidade pública de status, geração de links do WhatsApp e integridade do histórico.

O repositório do projeto no GitHub contém a documentação completa no arquivo README.md, detalhando arquitetura, pré-requisitos, passo a passo para execução e usuários cadastrados para demonstração.

Link do repositório no GitHub: [https://github.com/matheusgama/protetor\\_rf2](https://github.com/matheusgama/protetor_rf2) (ou conforme repositório informado na entrega).

## 7 RESULTADOS PRELIMINARES: SOLUÇÃO INICIAL

O sistema desenvolvido foi executado em ambiente local e validado com dados fictícios representativos da realidade de abrigo de animais. Abaixo são apresentadas as principais interfaces e resultados alcançados.

### 7.1 Área Pública e Vitrine de Adoção

A página inicial destaca a causa com métricas em tempo real e atalhos para a vitrine pública de cães e doações. O layout responsivo adapta-se perfeitamente de computadores para dispositivos móveis.

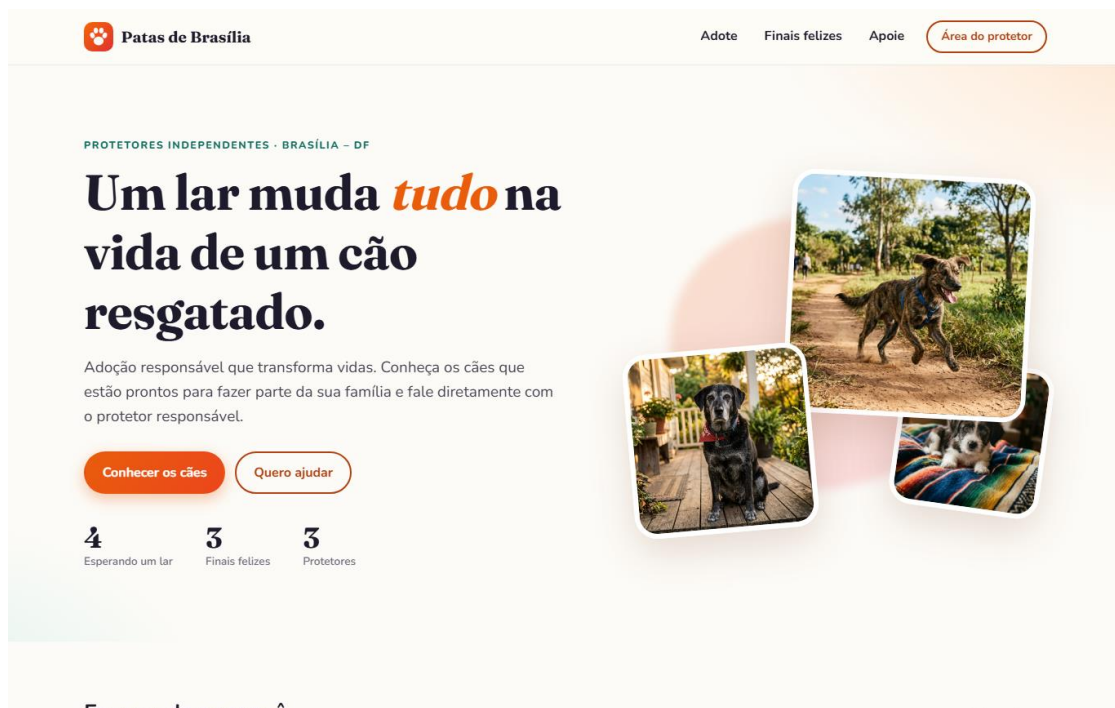


Figura: Página Inicial (Versão Desktop)



Figura: Página Inicial (Versão Mobile)

A vitrine pública (Figura 3 e Figura 4) permite a filtragem de animais por sexo, porte e idade, exibindo apenas cães com status elegíveis para adoção (RNF04 e RF04).

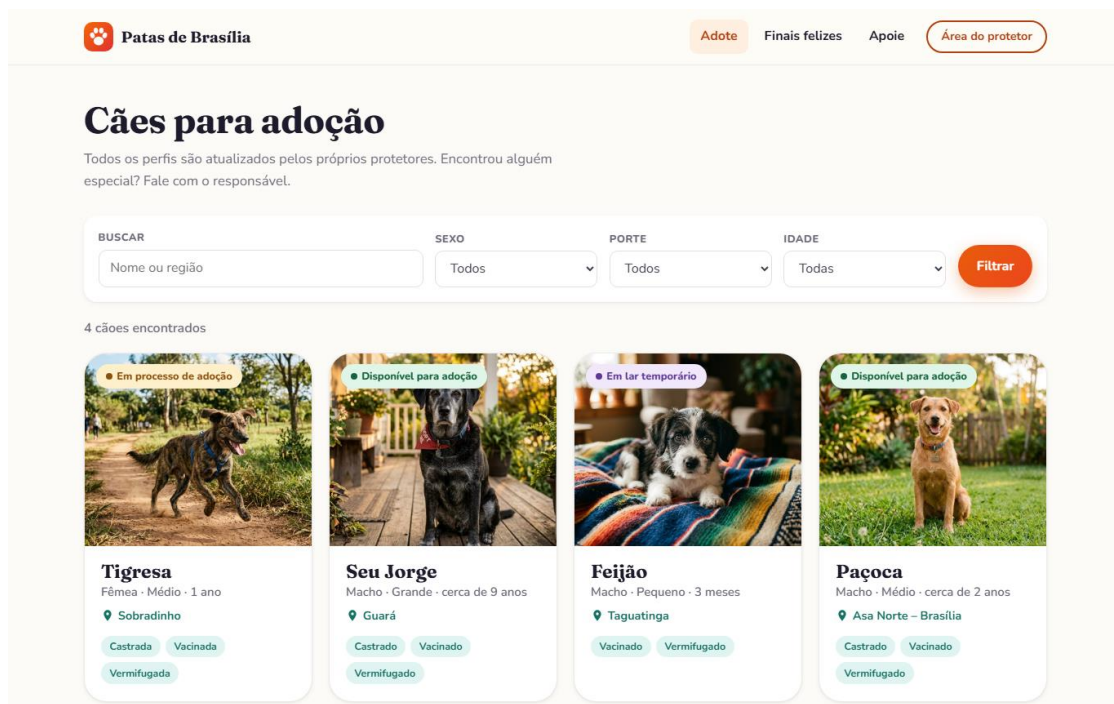


Figura: Vitrine Pública de Cães para Adoção com Filtros (Desktop)

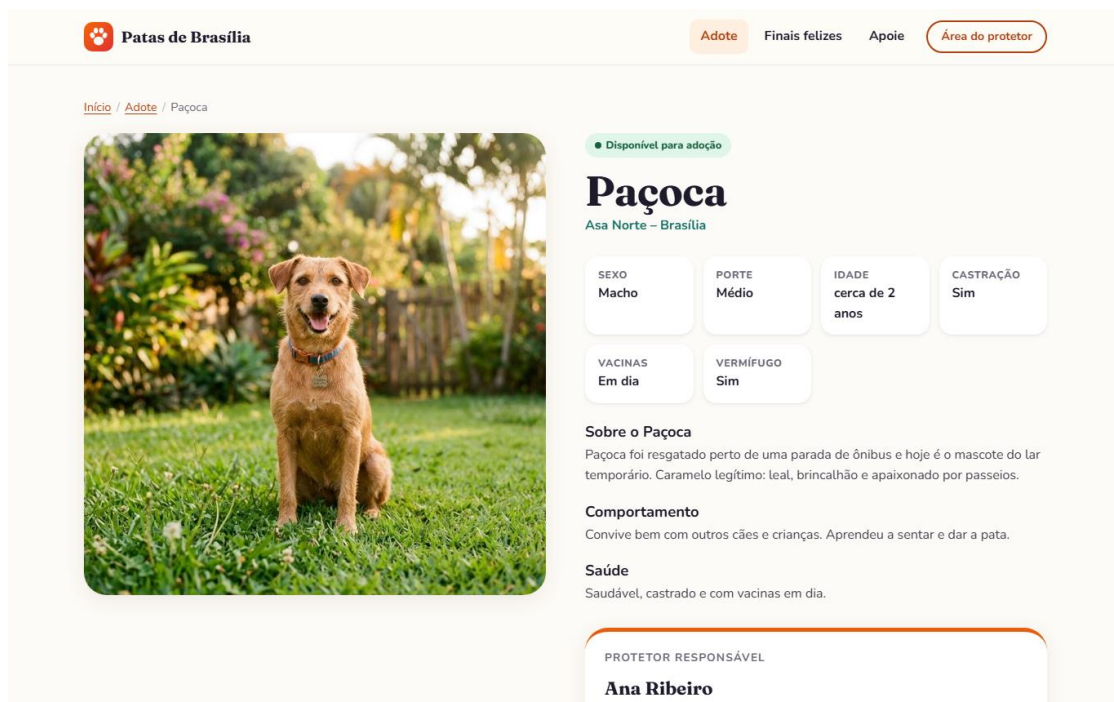


Figura: Página de Detalhe do Cão com Galeria de Fotos e Botão do WhatsApp (Desktop)



*Figura: Detalhes do Animal e Ação de Contato no Celular (Mobile)*

## 7.2 Área de Doações e Transparência

A seção 'Apoie' disponibiliza a chave PIX configurada pelo administrador, fornecendo um botão de cópia de chave e orientações transparentes para contribuição voluntária (RF07).

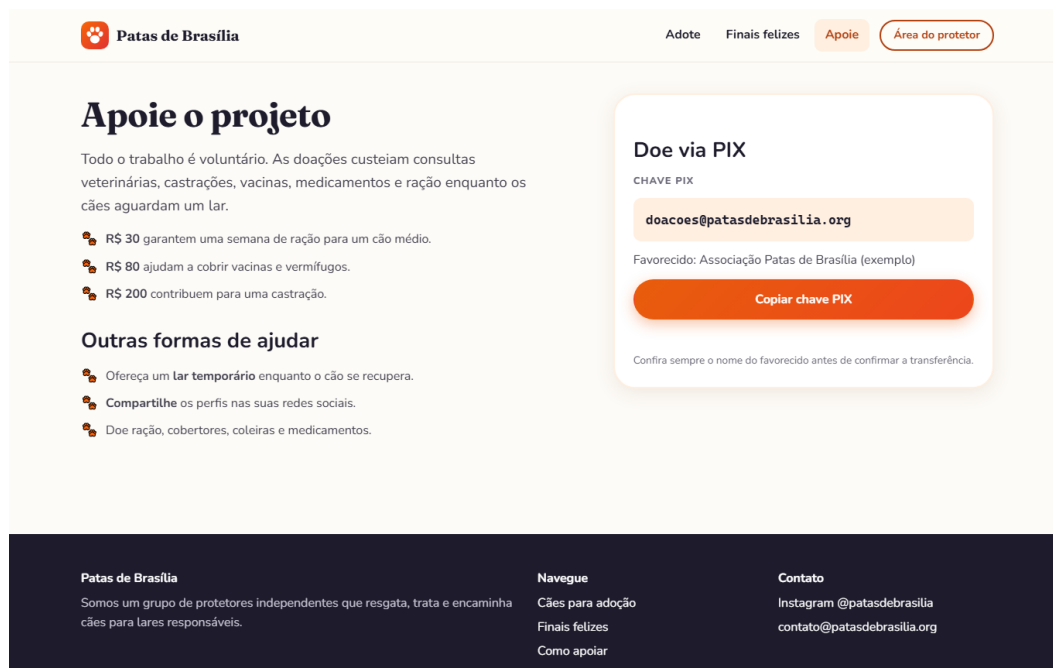


Figura: Seção de Doação com Chave PIX e Apoio Financeiro

### 7.3 Área Restrita dos Protetores e Gestão de Status

O acesso à área restrita (Figura 7) exige credenciais cadastradas. Após o login, o protetor visualiza o painel geral (Figura 8) com estatísticas por situação, lista de animais sob sua responsabilidade (RF05) e linha do tempo com histórico recente.

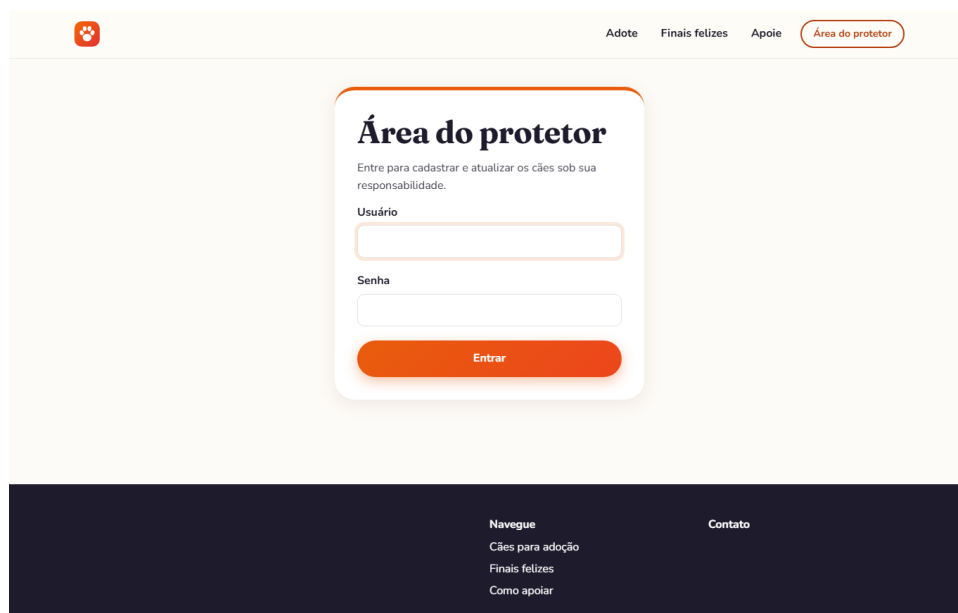


Figura: Tela de Login da Área Restrita do Protetor (RF01)

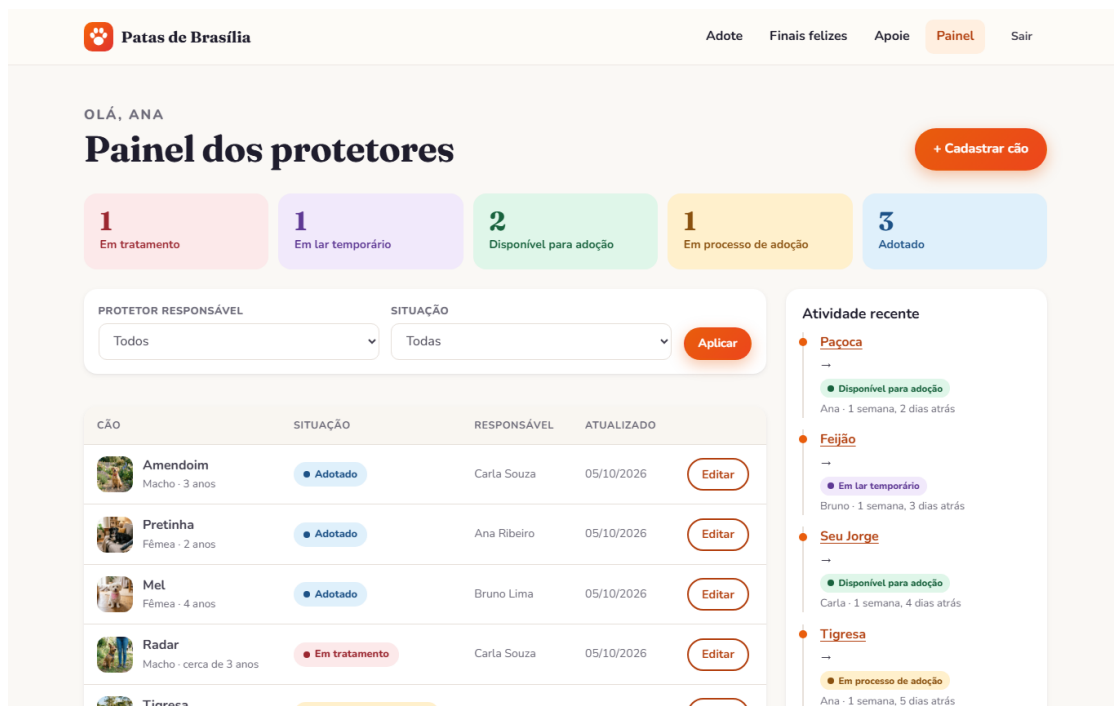


Figura: Painel de Controle com Indicadores e Filtros por Responsável (RF05)

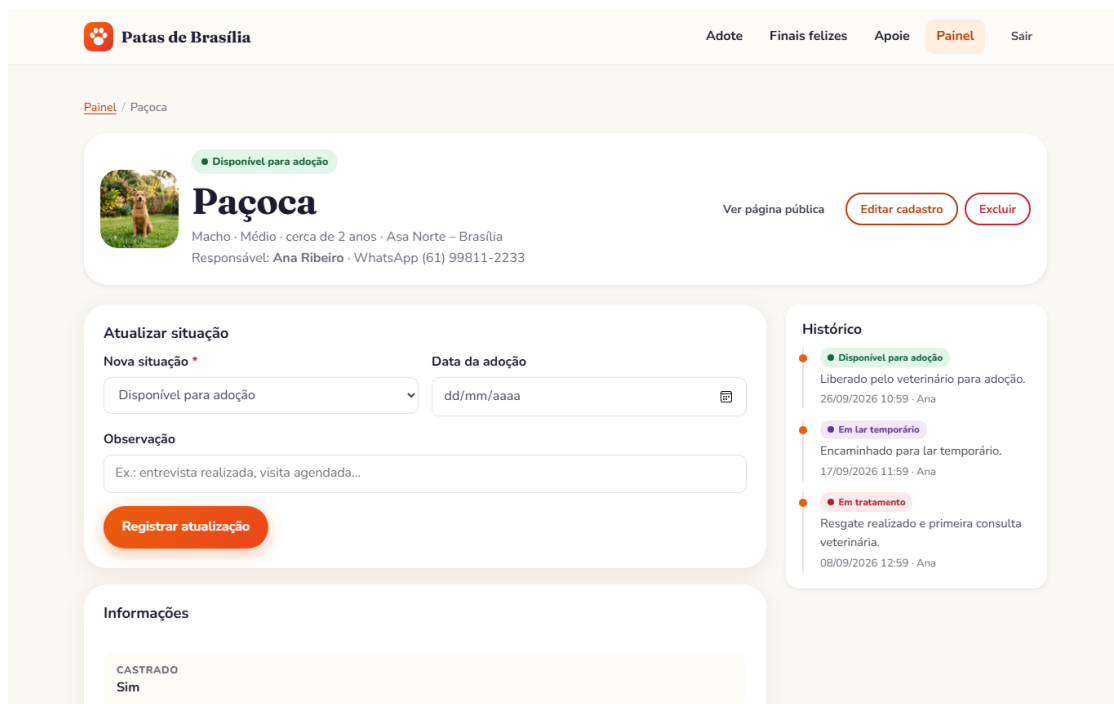


Figura: Ficha Interna do Animal, Atualização de Status e Histórico Automático (RF03 / RF08)

**Patatas de Brasília**

Adote Finais felizes Apoie **Painel** Sair

[Painel](#) / Novo cadastro

## Cadastrar cão

Campos marcados com \* são obrigatórios. Evite informar endereços completos ou dados pessoais de terceiros.

**Identificação**

Nome \*

Sexo \* Porte \*

Faixa etária \* Idade aproximada \*

Localização \*

Região onde o animal está (sem endereço completo).

**Responsável e situação**

*Figura: Formulário de Cadastro/Edição com Envio de Fotos (RF02)*

## 8 CONCLUSÃO

A entrega do Módulo 2 representou um avanço significativo no Projeto Integrador II, consolidando os requisitos levantados na fase analítica em uma solução web totalmente funcional, dinâmica e com elevado padrão de qualidade estético e técnico.

O uso do framework Django permitiu implementar com eficiência a lógica de negócios, controle de permissões e integridade de dados. A construção de uma interface responsiva baseada em HTML semântico e CSS3 mobile-first assegurou excelente usabilidade e acessibilidade em dispositivos móveis, essenciais para a rotina diária dos protetores no ambiente de resgate.

Como próximos passos para as etapas futuras do projeto, destacam-se a realização de testes de usabilidade diretamente com os protetores voluntários, a inclusão de funcionalidade para geração automática de imagens formatadas para stories das redes sociais e a implantação da aplicação em ambiente de nuvem.

## REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

DJANGO SOFTWARE FOUNDATION. Django documentation: Version 5.2. 2026. Disponível em: <https://docs.djangoproject.com/>. Acesso em: 05 out. 2026.

ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018: Systems and software engineering - Life cycle processes - Requirements engineering. Geneva: ISO, 2018.

ISO/IEC. ISO/IEC 25010:2017: Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models. Geneva: ISO, 2017.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2019.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education do Brasil, 2015.

W3C. Web Content Accessibility Guidelines (WCAG) 2.1. 2023. Disponível em: <https://www.w3.org/TR/WCAG21/>. Acesso em: 05 out. 2026.